

Day 18: Create and Update Profile Dictionary

1. Day Number

Day 18

2. Topic Name

Dictionaries

3. Connection

Earlier, you cleaned user details such as the name and phone number.

Today, you will store related user information together inside a **dictionary**.

For example:

```
name = "Rahul"  
phone = "9876543210"  
city = "Bengaluru"
```

Instead of keeping three separate variables, you can store them together:

```
profile = {  
    "name": "Rahul",  
    "phone": "9876543210",  
    "city": "Bengaluru"  
}
```

4. Important Topics

Key-value pair

A dictionary stores information as **key-value pairs**.

```
"name": "Rahul"
```

Here:

- "name" is the **key**
- "Rahul" is the **value**

The key describes what the value represents.

Accessing values

You can access a value using its key:

```
print(profile["name"])
```

Output:

```
Rahul
```

Updating values

You can change an existing value by assigning a new value to its key:

```
profile["city"] = "Mumbai"
```

The old city will be replaced with the new city.

Using .get()

The .get() method safely reads a dictionary value:

```
print(profile.get("phone"))
```

A useful difference is:

```
profile["country"]
```

This causes an error when "country" does not exist.

But:

```
profile.get("country")
```

returns:

```
None
```

You can also provide a default message:

```
profile.get("country", "Not available")
```

5. Foundational Notes

What is a dictionary?

A dictionary is a Python data structure that stores related information using named keys.

Basic syntax:

```
dictionary_name = {  
    "key1": "value1",  
    "key2": "value2"  
}
```

Important dictionary rules

- A dictionary uses curly brackets: { }
- Every key is connected to a value using :
- Key-value pairs are separated by commas

- Keys should normally be unique
- Values can be strings, numbers, lists, or other data types

Example:

```
student = {  
    "name": "Anita",  
    "age": 22,  
    "passed": True  
}
```

6. Easy Example

This example is separate from today's profile problem:

```
book = {  
    "title": "Python Basics",  
    "pages": 150  
}  
  
print(book.get("title"))  
  
book["pages"] = 175  
  
print(book)
```

Output:

```
Python Basics  
{'title': 'Python Basics', 'pages': 175}
```

What happened?

1. A dictionary named book was created.
2. The title was accessed using `.get()`.
3. The number of pages was updated.
4. The updated dictionary was printed.

7. Problem Statement

Create a profile dictionary containing:

- Name
- Phone
- City

Ask the user to enter a new city.

Update the city value inside the profile dictionary.

Finally, print the updated profile.

Example starting profile:

Name: Rahul
Phone: 9876543210
City: Bengaluru

Example user update:

Enter new city: Hyderabad

The dictionary should now contain Hyderabad as the city.

8. Concepts Used

You will use:

- Dictionary creation
 - Key-value pairs
 - String keys
 - Accessing dictionary values
 - Updating dictionary values
 - `.get()` method
 - `input()`
 - `strip()`
 - Function
 - Parameter
 - Return value
 - `if` condition
-

9. Thought Process

Step 1: Create the profile

Create one dictionary containing the user's name, phone, and current city.

Think of it like a profile card:

```
profile
├── name   → Rahul
├── phone  → 9876543210
└── city   → Bengaluru
```

Step 2: Ask for the new city

Use `input()` to ask the user for a city.

Clean the input using `.strip()` so extra spaces are removed.

Step 3: Check the city

If the user enters a proper city, update the "city" key.

If the input is empty, keep the existing city unchanged.

Step 4: Return the profile

Because the suggested approach is functional, create a function that receives:

- The profile dictionary
- The new city

The function should update the city when valid and return the profile.

Step 5: Print the result

Print the updated dictionary or print its values one by one.

10. Pseudocode

```
START

CREATE a profile dictionary containing:
    name
    phone
    city

DEFINE a function that receives profile and new city

    CLEAN the new city by removing extra spaces

    IF new city is not empty
        UPDATE the city value in profile
    ELSE
        KEEP the old city unchanged

    RETURN profile

ASK the user to enter a new city

CALL the function using profile and entered city

STORE the returned profile

PRINT the updated profile

END
```

11. Suggested Solving Approach: Functional Approach

Create a function with a responsibility similar to:

```
update_city(profile, new_city)
```

The function should:

1. Receive the profile dictionary.
2. Receive the new city.

3. Clean the city input.
4. Check whether it is empty.
5. Update `profile["city"]` when valid.
6. Return the profile.

Suggested program structure:

```
Create profile dictionary
    ↓
Ask for new city
    ↓
Call update function
    ↓
Receive updated dictionary
    ↓
Print profile
```

12. Easy Edge Cases

Empty city

Input:

```
Enter new city:
```

The program should not replace the existing city with an empty string.

Possible behaviour:

```
City cannot be empty.
```

The original city should remain unchanged.

City with extra spaces

Input:

```
Hyderabad
```

Use `.strip()` so the stored value becomes:

```
Hyderabad
```

Missing key idea

Suppose the dictionary does not contain "country".

Using:

```
profile["country"]
```

can cause a `KeyError`.

Using:

```
profile.get("country")
```

is safer.

You may also provide a default value:

```
profile.get("country", "Not provided")
```

For today's problem, the "city" key should already exist.

13. Expected Input

Example:

```
Enter new city: Hyderabad
```

The starting profile values may already be written in the program:

```
Name: Rahul  
Phone: 9876543210  
City: Bengaluru
```

14. Expected Output

One possible output:

```
Updated profile:  
Name: Rahul  
Phone: 9876543210  
City: Hyderabad
```

You may also print the complete dictionary:

```
{'name': 'Rahul', 'phone': '9876543210', 'city': 'Hyderabad'}
```

For an empty city, a possible output is:

```
City cannot be empty.  
City remains: Bengaluru
```

15. Hint Only

Create a dictionary using {} and three keys:

```
"name"  
"phone"  
"city"
```

Inside your function:

- Clean the new city using `.strip()`
- Check whether the cleaned city contains text
- Update the city using the "city" key
- Return the profile dictionary

To safely display a value, try:

```
profile.get("city", "City not found")
```

Do not create a second profile dictionary unnecessarily. Update and return the dictionary received by the function.